Shell Sort e Selection Sort

Funcionamento, implementação e análise de desempenho



Iniciar

História do Shell Sort

TRÊS NOMES PARA LEMBRAR

1959

Donald L. Shell

Publicou o Shell Sort como uma melhoria da ordenação por inserção.

1973

Donald E. Knuth

Apresentou a sequência de gaps gerada por 3h + 1.

2001

Marcin Ciura

Publicou um estudo experimental de sequências para melhorar o desempenho médio.

Shell Sort aproxima os valores antes da última passagem.

Como funciona o Shell Sort

ORDENA PRIMEIRO OS VALORES MAIS DISTANTES

1 **Separar em grupos**

O gap define a distância entre as posições de cada grupo.

2 **Ordenar cada grupo**

Como na inserção: desloca os maiores e encaixa o valor.

3 **Diminuir o gap e repetir**

Com gap 1, todos ficam no mesmo grupo e a ordenação termina.

Os primeiros gaps aproximam os valores das posições finais.

Gaps e sua importância

O GAP É A DISTÂNCIA ENTRE AS POSIÇÕES

A escolha muda as passagens, os deslocamentos e o tempo.

Metades

Começa em $\lfloor n/2 \rfloor$ e divide por dois, com divisão inteira, até 1.

Para n = 100: 50, 25, 12, 6, 3, 1.

Knuth

1, 4, 13, 40, 121, 364, ...

Próximo gap = 3h + 1.

Ciura fixo

1, 4, 10, 23, 57, 132, 301, 701, 1750

Não usamos a versão estendida.

Usamos do maior para o menor, terminando em gap 1.

Shell Sort: execução passo a passo

GAPS POR METADES • 4, 2 E 1

Entrada

8	3	7	4	6	2	5	1

Resultado

1	2	3	4	5	6	7	8

```
for (size_t gap = n / 2; gap > 0; gap /= 2) {
  for (size_t i = gap; i < n; ++i) {
    int tmp = v[i];
    size_t j = i;
    while (j >= gap && v[j - gap] > tmp) {
      v[j] = v[j - gap];
      j -= gap;
    }
    v[j] = tmp;
  }
}
```

Na web: reproduza, pause e avance com a linha do código destacada.

O PDF contém este resumo estático. A execução animada está no HTML.

Shell Sort: por subsequências

OUTRA FORMA DE PERCORRER OS GRUPOS DO MESMO GAP

Gap 4: cada linha é um grupo.

Grupo 0	**8** índice 0	**6** índice 4
Grupo 1	**3** índice 1	**2** índice 5
Grupo 2	**7** índice 2	**5** índice 6
Grupo 3	**4** índice 3	**1** índice 7

```
for (size_t gap = n / 2; gap > 0; gap /= 2) {
  for (size_t r = 0; r < gap; ++r) {
    for (size_t i = r + gap; i < n; i += gap) {
      int tmp = v[i];
      size_t j = i;
      while (j >= gap && v[j-gap] > tmp) {
        v[j] = v[j-gap];
        j -= gap;
      }
      v[j] = tmp;
    }
  }
}
```

Ordenamos um grupo inteiro antes de passar para o próximo.

Na web: inserção animada em cada grupo, com os índices do vetor original.

Shell Sort é estável?

NÃO GARANTE A ORDEM ORIGINAL DOS VALORES IGUAIS

2A e 2B têm o mesmo valor. As letras identificam cada elemento.

Entrada	**2A**	**2B**	**1**	**3**
Após gap 2	**1**	**2B**	**2A**	**3**
Após gap 1	**1**	**2B**	**2A**	**3**

O salto do gap 2 colocou 2A depois de 2B.

Mesmo com a inserção normal no final, essa ordem pode permanecer.

Complexidades das implementações

MELHOR E PIOR CASO DE CADA VERSÃO

Versão	Melhor caso	Pior caso
Metades	$\Theta(n \log n)$	$\Theta(n^2)$
Knuth	$\Theta(n \log n)$	$\Theta(n^{3/2})$
Ciura fixo até 1750	$\Theta(n)$	$\Theta(n^2)$

Ciura: o melhor caso linear vale para a nossa lista fixa, com no máximo nove passagens.

Por que a análise do Shell é difícil?

EXISTEM PROVAS, MAS NEM TODOS OS CASOS ESTÃO RESOLVIDOS

1 **As passagens dependem umas das outras**

Cada gap reorganiza o vetor e muda o trabalho da passagem seguinte.

2 **A sequência de gaps faz diferença**

Já existem provas de melhor e pior caso para várias sequências.

3 **O caso médio ainda traz desafios**

Ainda faltam limites precisos para várias sequências usadas na prática.

Poonen (1993): limite inferior do pior caso

$$\Omega\left(n \left(\frac{\log n}{\log \log n}\right)^2\right)$$

Para qualquer sequência de gaps, existe uma entrada que exige pelo menos essa ordem de operações.

Logo, o Shell convencional não pode garantir pior caso O(n log n).

Shell: vantagens e desvantagens

POUCA MEMÓRIA EXTRA, MAS O DESEMPENHO DEPENDE DOS GAPS

Vantagens

Memória auxiliar constante.

Move valores distantes
nas primeiras passagens.

Permite escolher diferentes
sequências de gaps.

Desvantagens

Não é estável.

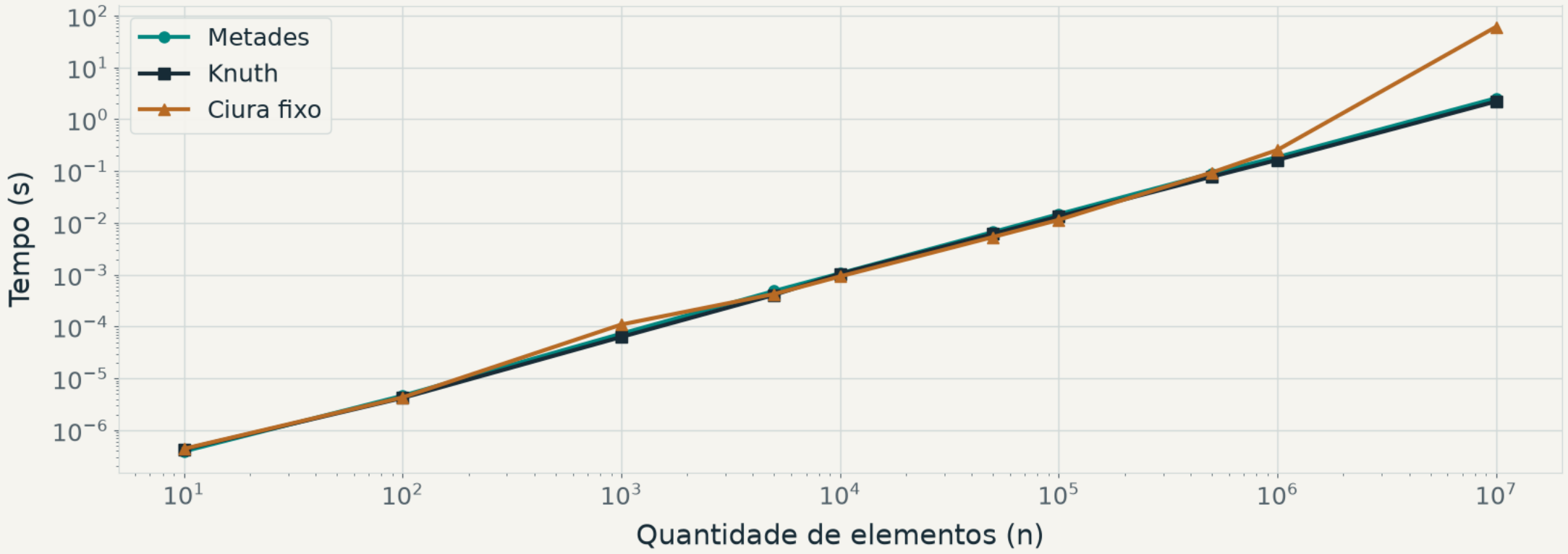
O desempenho depende
da sequência de gaps.

Algumas sequências têm
pior caso quadrático.

A análise do pior caso é mais difícil que a do Selection Sort.

Tempo — entrada aleatória

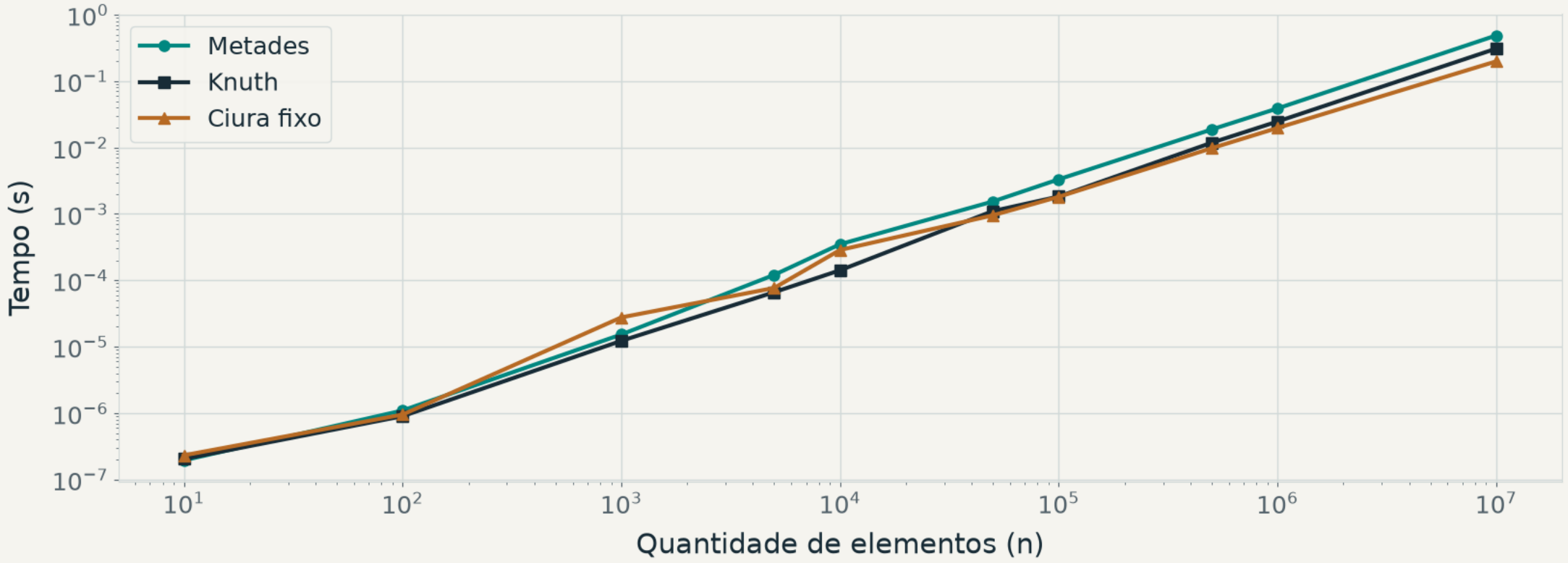
SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Tempo — entrada crescente

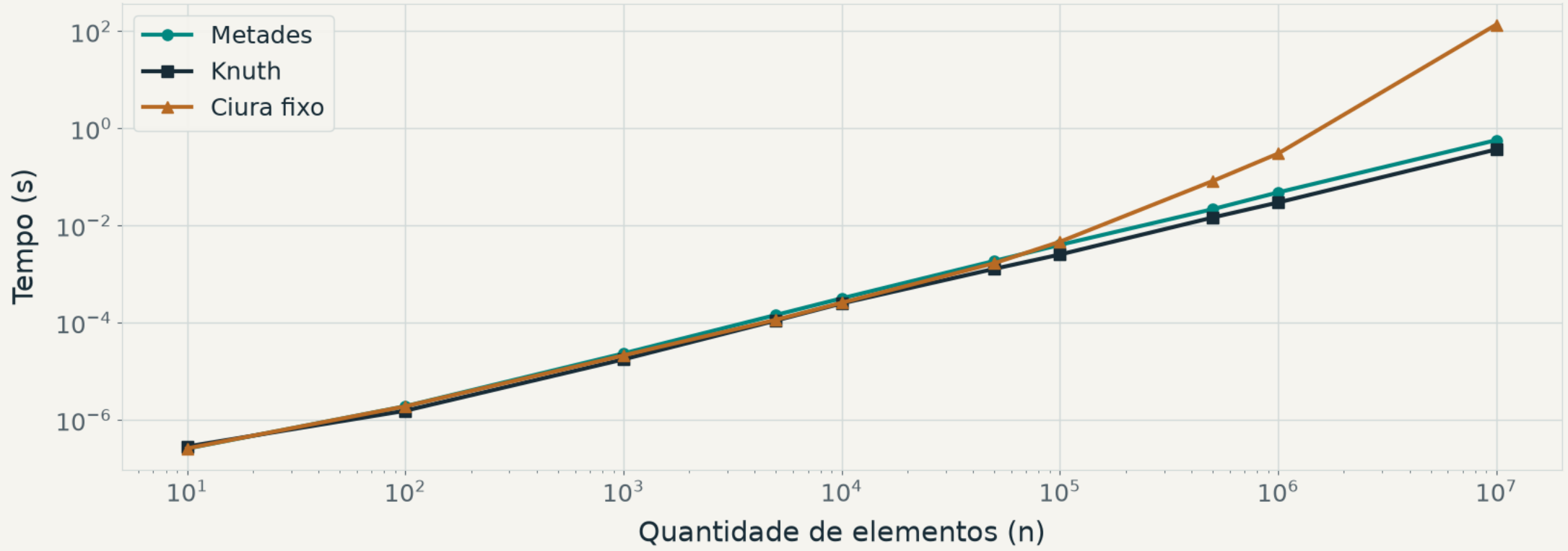
SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Tempo — entrada decrescente

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Entrada decrescente não é, automaticamente, o pior caso matemático.

Conclusão: a escolha dos gaps

NENHUMA SEQUÊNCIA VENCEU TODOS OS CENÁRIOS

Knuth

Foi o mais rápido no teste decrescente com 10 milhões de elementos.

Ciura fixo

Foi mais rápido nas entradas crescentes maiores, mas teve alto custo na decrescente. Sua lista termina em 1750.

Tamanho e ordem da entrada

Os resultados mudaram conforme o tamanho do vetor e sua organização.

Uma medição por teste: repetir ajudaria a avaliar a variação dos tempos.

Os resultados medidos não são uma garantia para qualquer entrada.

O que lembrar sobre a história?

Época	O que sabemos
Anos 1950	Já existiam estudos de métodos de seleção.
1956 e 1958	Pesquisadores publicaram trabalhos de ordenação.
Anos 1970–1990	O assunto aparece em livros de algoritmos.
Ano 2000	Selection Sort já era ensinado em cursos, como no MIT.

Não há um inventor comprovado do Selection Sort.

Shell Sort, publicado por Donald Shell em 1959, é outro algoritmo.

Selection Sort: execução passo a passo

BUSCAR O MENOR • TROCAR • REPETIR

Entrada

64	25	12	22	11

Resultado

11	12	22	25	64

```
for (size_t i = 0; i + 1 < n; ++i) {
  size_t menor = i;
  for (size_t j = i + 1; j < n; ++j) {
    if (v[j] < v[menor]) {
      menor = j;
    }
  }
  if (menor != i) {
    int tmp = v[i];
    v[i] = v[menor];
    v[menor] = tmp;
  }
}
```

Na web: vetor, código e índices animados. No PDF: resumo estático.

Melhor caso e pior caso: $\Theta(n^2)$

A CONTAGEM DE COMPARAÇÕES NÃO DEPENDE DA ORDEM DOS VALORES

$$C(n) = (n - 1) + (n - 2) + \cdots + 1 = n(n - 1) / 2$$

Melhor caso

Entrada já crescente: não ocorre troca. Mas o laço interno continua fazendo todas as comparações.

Pior caso

Mesmo com entrada desfavorável, a contagem de comparações continua sendo $n(n - 1)/2$.

$O(n^2) + \Omega(n^2) \Rightarrow \Theta(n^2)$ em ambos os casos

Estabilidade e uso de memória

A VERSÃO COM TROCA DIRETA NÃO É ESTÁVEL

Entrada



Após a 1ª troca



A ordem de 2A e 2B foi invertida.

Ordenação: Θ(1) de memória extra • Até n − 1 trocas

O programa completo também reserva um vetor de entrada com malloc(): Θ(n) de armazenamento.

Vantagens

Vantagens e desvantagens

SIMPLES DE ENTENDER, MAS CUSTOSO PARA ENTRADAS GRANDES

Vantagens

- Código curto e fácil de explicar
- Memória auxiliar $\Theta(1)$
- No máximo $n - 1$ trocas efetivas

Desvantagens

- $\Theta(n^2)$ mesmo se a entrada já está em ordem
- Não preserva a ordem dos iguais
- Tempo elevado para vetores grandes

Menos trocas não garantem menor tempo: o programa continua procurando o menor.

Computador utilizado nos testes

DELL OPTIPLEX 3060

Processador	Intel Core i5-8500T, 8ª geração
Frequência base	2,10 GHz
Núcleos e threads	6 núcleos e 6 threads
Memória RAM	16 GB — 2 módulos de 8 GB
Tecnologia da RAM	DDR4, 2400 MT/s
Armazenamento	SSD NVMe ADATA SM2P41C3, 256 GB
Sistema operacional	Ubuntu Server

Configuração informada no relatório do trabalho.

Selection Sort × Shell Sort

COMPARAÇÃO COM SHELL POR METADES

Selection Sort

Melhor caso: $\Theta(n^2)$
Pior caso: $\Theta(n^2)$
Busca o menor a cada rodada.

Shell por metades

Melhor caso: $\Theta(n \log n)$
Pior caso: $\Theta(n^2)$
Aproveita a organização dos gaps.

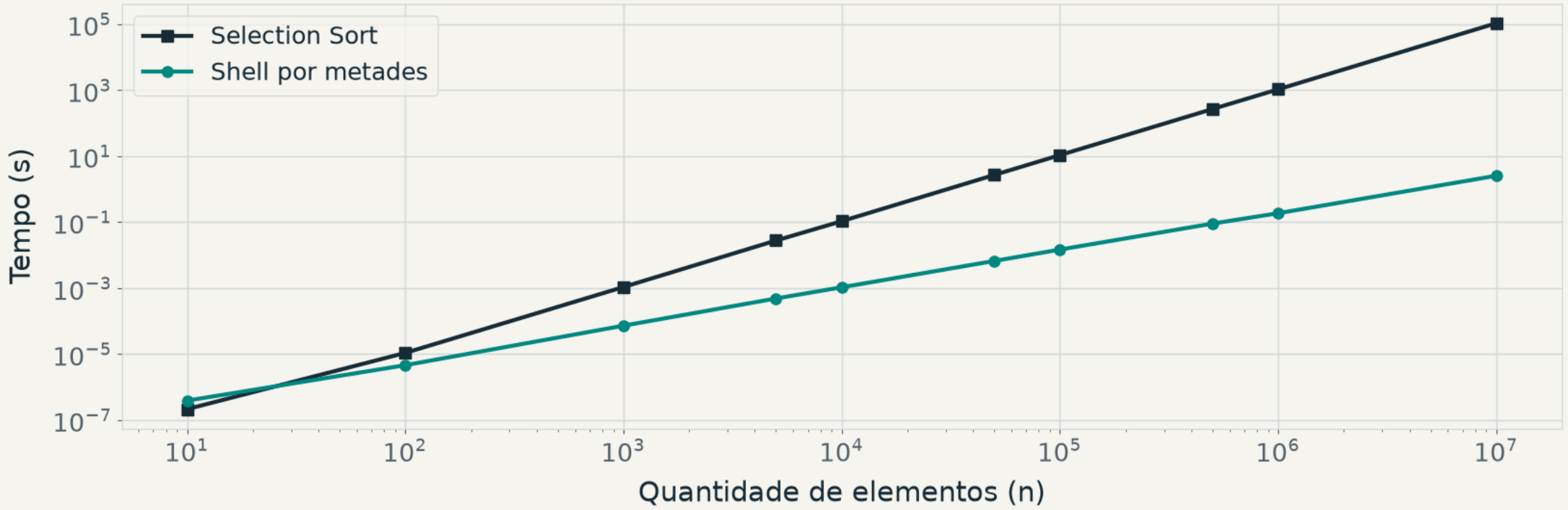
Ambos: memória auxiliar constante e sem garantia de estabilidade.

Mesmo pior caso não significa o mesmo tempo em todos os testes.

Nesta comparação, usamos somente Selection e Shell por metades.

Tempos — entrada aleatória

SELECTION SORT × SHELL POR METADES



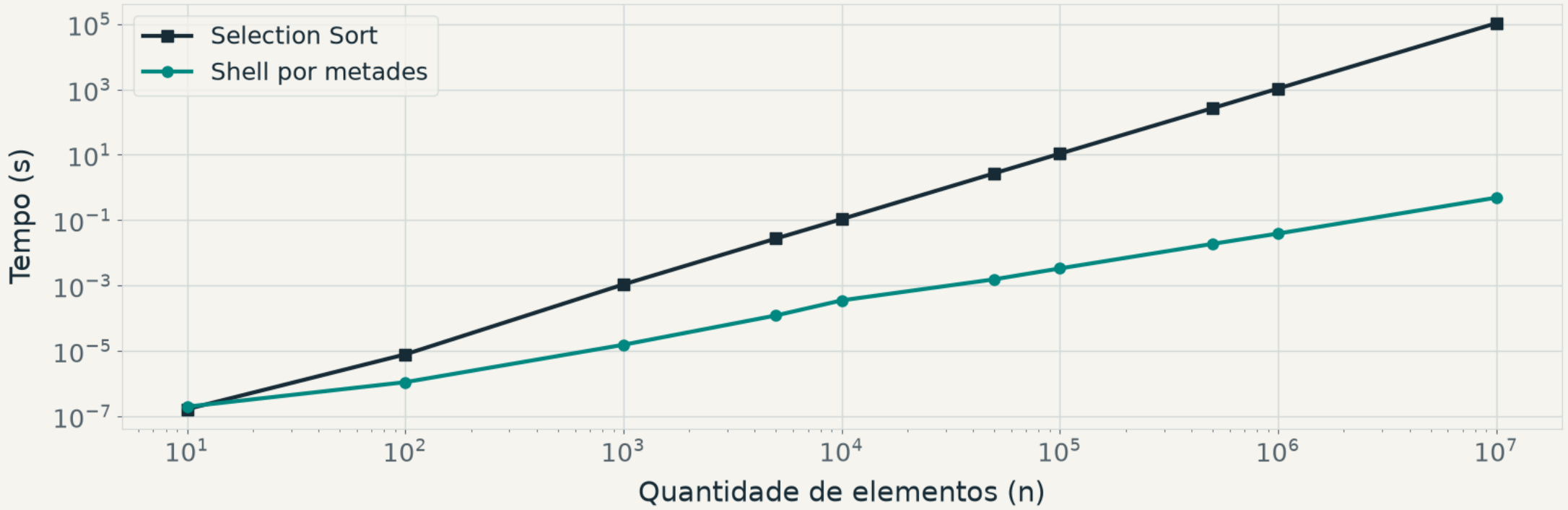
10 milhões de elementos:

Selection: 107.310,582 s Shell por metades: 2,559 s

Escalas logarítmicas. Medições individuais do relatório, sem médias.

Tempos — entrada crescente

SELECTION SORT × SHELL POR METADES



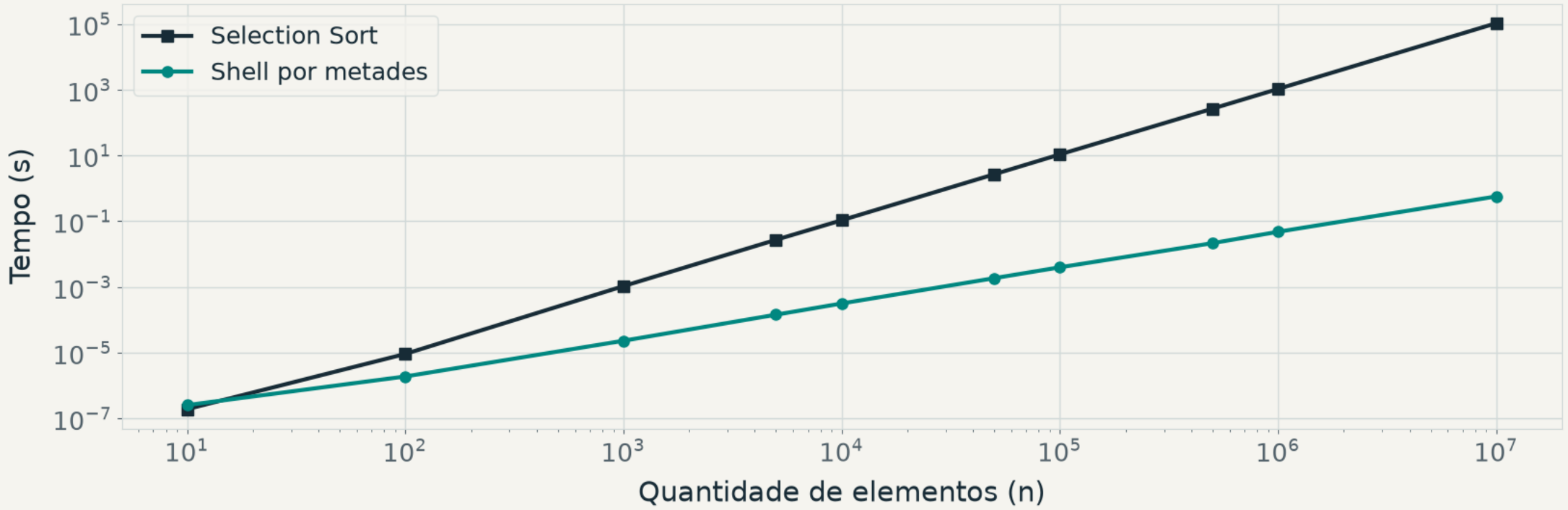
10 milhões de elementos:

Selection: 105.912,840 s Shell por metades: 0,487 s

Escalas logarítmicas. Medições individuais do relatório, sem médias.

Tempos — entrada decrescente

SELECTION SORT × SHELL POR METADES



10 milhões de elementos:

Selection: 105.923,688 s Shell por metades: 0,570 s

Escalas logarítmicas. Medições individuais do relatório, sem médias.

Menos escritas não garantem menor tempo

10 MILHÕES DE ELEMENTOS • ENTRADA ALEATÓRIA

Selection Sort

Comparações

49.999.995.000.000

Escritas no vetor

19.999.976

Shell por metades

Comparações

1.113.901.611

Escritas no vetor

1.118.919.386

Shell escreveu mais, mas fez muito menos comparações.

Na entrada crescente, Selection não fez trocas,
mas continuou fazendo todas as buscas.

Conclusão: Selection e Shell

O QUE OBSERVAMOS NOS TESTES

1 **Shell por metades foi mais rápido nos vetores grandes**

Com 10 milhões aleatórios: quase 30 horas contra cerca de 2,559 s.

2 **Selection não aproveitou a entrada já ordenada**

Sem trocas, mas com a mesma quantidade de comparações.

3 **O pior caso não descreve todas as entradas**

Ambos têm pior caso quadrático, mas os tempos medidos foram diferentes.

Precisamos olhar tempo, comparações e escritas em conjunto.

Bibliografia do Shell Sort

REFERÊNCIAS • ALGORITMO E SEQUÊNCIAS DE GAPS

Donald E. Knuth

The Art of Computer Programming, volume 3: Sorting and Searching.

Marcin Ciura (2001)

Estudo experimental de sequências de gaps.

Robert Sedgewick (1986 e 1996)

Limites e referências para outras sequências.

Abrir também o estudo de 1996

Resultados: shell_metades, shell_knuth e shell_ciura (CSV).

Uma medição por combinação. Sem execução nova nem médias.

Comparação e conclusão: relatório do trabalho, seções 4 e 7.

Fontes: análise do Shell Sort

REFERÊNCIAS • CASO MÉDIO E PIOR CASO

VITÁNYI, Paul (2018)

On the average-case complexity of Shellsort.

Random Structures & Algorithms, v. 52, n. 2, p. 354–363.

DOI: 10.1002/rsa.20737 PDF no CWI

JIANG, Tao; LI, Ming; VITÁNYI, Paul

A Lower Bound on the Average-Case Complexity of Shellsort.

arXiv:cs/9906008 (1999); publicado no JACM em 2000.

arxiv.org/abs/cs/9906008

POONEN, Bjorn (1993)

The Worst Case in Shellsort and Related Algorithms.

Journal of Algorithms, v. 15, n. 1, p. 101–124.

math.mit.edu/~poonen/papers/shell.pdf